매뉴얼 PDF 한글화 노하우

PDF 번역은 글을 뽑아 새 문서를 만드는 일이 아니다. 원본 쪽의 위치·크기·굵기·삽화는 그대로 두고 **영문만 한국어로 바꾸는 일**이다. 파일이 열리고 검색된다고 끝난 게 아니다. **원본과 나란히 놓고 모든 쪽을 눈으로 본 뒤**에야 끝난다.

방식 1

로컬 AI 방식

GPU가 있는 PC에서 ollama 비전·번역 모델과 ComfyUI 인페인팅으로. 비용은 전기뿐이고 문서가 밖으로 나가지 않는다.

방식 2

API 방식

로컬 AI 없이 OCR·LLM API와 인페인팅 API·서비스로. 코딩 없이 채팅 AI와 편집기로 하는 길도 함께.

두 방식 모두 먼저 공통 원리를 따른다. 다른 AI에 맡길 때는 방식별 '작업 지시서'와 공통 원리, 그 방식의 규칙을 함께 붙여 넣으면 작업 지시가 된다.

공통

공통 원리 — 어떤 도구를 쓰든 같은 것

로컬 모델을 쓰든 API를 쓰든 PDF를 한글판으로 만드는 순서와 판단 기준은 같다. 아래 규칙은 두 방식 모두에 그대로 적용한다.

원칙

- ★ 원본 쪽의 위치·크기·굵기·정렬·삽화는 두고 영문만 한국어로 바꾼다.
- ★ 한글은 이미지로 구워 넣지 않고 실제 텍스트로 넣는다. 선택·검색이 돼야 한다.
- ★ 완료 기준은 원본|결과를 나란히 렌더한 모든 쪽을 눈으로 확인한 것이다.
- – 원본은 건드리지 않는다. 단계마다 사람이 고칠 수 있는 중간 파일(JSON·오버레이 이미지)을 남긴다.
- – AI 모델은 양(글자 읽기·초벌 번역·바탕 채움)을 맡는다. 구조 판단(단 경계·번역 단위·표 처리)과 최종 검수는 사람이나 상위 모델이 맡는다.

쪽마다 문서 유형부터 가른다

같은 PDF 안에서도 쪽마다, 한 줄 안에서도 유형이 섞인다. 쪽별 추출 글자 수와 폰트 목록을 뽑는다. 0자이거나 폰트가 이상한 쪽은 렌더해서 본다.

```
import fitz  # PyMuPDF
for i, p in enumerate(fitz.open(src)):
    print(i + 1, len(p.get_text().strip()), sorted({f[3] for f in p.get_fonts()}))
```

유형	판별	위치의 기준	지우는 법
A **벡터 글자**	글자 수 > 0, 복사한 글이 화면과 같다	PDF가 가진 좌표·폰트·크기 그대로	텍스트 객체만 삭제(redaction, 이미지·도형 제외). 삽화는 1픽셀도 안 바뀐다
B **그림 속 글자**	글자 수 0, 또는 A를 지운 렌더에 글자가 남는다 (포스터·손글씨·아이콘 라벨·지도)	잉크 행(글자 획의 연결 성분)	잉크 마스크만 채운다

유형	판별	위치의 기준	지우는 법
C **스캔 + 숨은 층**	글자 수 > 0인데 보이지 않는 폰트(Courier·재인코딩)이고 줄이 화면과 어긋난다	잉크 행. 숨은 층은 글 내용의 힌트	번역 블록 상자 안의 어두운 픽셀을 채운다

⚠ 숨은 OCR 층은 줄을 통째로 빠뜨리고(20% 안팎), 이웃 단을 한 줄로 잇고, 이전 쪽 글을 품기도 한다. 위치의 기준으로 쓰지 않는다.

⚠ 숨은 층의 줄과 새로 읽은 줄을 한 목록에 섞지 않는다. 한 줄씩 어긋나 유령 줄이 생기고 번역이 두 번 찍힌다. 단마다 출처는 하나만 쓴다.

– 숨은 층의 글로 빈칸을 메울 때는 새로 읽은 글과 가장 닮은 조각 하나만(길이 1.25배 이내) 쓴다. 여러 조각을 이으면 옆 단 글이 섞인다.

⚠ 추출 순서는 믿지 않는다. 좌우 단·도해 사이에서 섞여 번역이 한 칸씩 밀린다. 읽기 순서는 좌표로 정한다.

⚠ 두 단 조판은 단 사이 간격이 보통 띄어쓰기와 같아 한 줄로 이어진 채 추출되기도 한다. 단 경계는 빈 골(세로 투영)로 후보를 찾고, 문장이 실제로 이어지는지 읽어서 확정한다.

⚠ 일부 재인코딩 폰트는 글자 보존 플래그(PyMuPDF `PRESERVE_*`)를 주면 글자가 U+FFFD로 나온다. 기본 플래그로 추출한다.

결과물 형태를 먼저 정한다

	원본 위 재조판	새로 조판
결과	원본 쪽 위 같은 자리에 한글	원본 판형·삽화만 살리고 본문을 새로 흘린다. 원문 쪽과 번역을 나란히 놓는 대조판도 있다
맞는 문서	삽화와 글이 쪽마다 엮인 매뉴얼·저널·레퍼런스 카드	글 위주의 긴 매뉴얼
번역 단위	블록. 쪽을 넘는 문장은 사슬로	쪽 또는 절
어려운 점	문서마다 단 경계·제외 영역 손질, 표	삽화 둘레로 잘게 끊긴 문장을 소제목으로 오인하기 쉽다

어느 쪽이든 번역문 정본은 하나(원문 키로 묶인 JSON)로 두고, 결과물만 따로 만든다.

공정 — 7단계와 중간 파일

단계마다 JSON을 남기고, 다음 단계로 넘어가기 전에 오버레이 이미지로 확인한다. 사람이 JSON을 고치면 다음 단계는 고친 값을 쓴다.

1 **`analyze`** 쪽별 유형 판정, 벡터 줄 상자(베이스라인·폭·폰트·크기·굵기) 실측, 단 경계 후보. 산출: `analysis.json`, 쪽별 오버레이.

단 경계는 사람이 확정한다.

2 **`ocr`** B·C 쪽의 글자 읽기와 잉크 행 검출. 산출: `ocr.json`, 마스크 PNG.

마스크가 글자만 덮고 삽화를 안 건드리는가, 행 상자가 한 줄씩인가.

3 **`blocks`** 번역 단위 확정 — 역할(제목·본문·라벨·캡션·명단), 끊긴 문장 사슬. 산출: `blocks.json`.

두 단이 한 블록으로 묶이지 않았는가(바로 앞 줄과도 가로로 겹쳐야 같은 블록), 표가 문단으로 묶이지 않았는가.

4 **`translate`** 사슬은 한 단위로 번역. 산출: `blocks_ko.json` (번역과 플래그).

빈 번역·혼입 문자·미번역·길이 초과 플래그부터 본다.

5 **`erase`** 벡터는 텍스트 객체만, 래스터는 잉크 마스크만 지운다. 산출: `erased.pdf`.

영문 잔존, 삽화 훼손.

6 **`typeset`** 원본 줄 상자에 한글을 흘린다. 산출: `out.pdf`, 조판 보고서.

넘침 0, 축소율 0.85 미만 블록 목록.

7 **`qa`** 원본|결과를 나란히 렌더한다. 산출: 쪽별 비교 이미지, 문제 목록.

전 쪽 확인. 문제는 원인별로 묶어 고친다.

`blocks.json` 한 항목의 모양:

```
{
  "id": "p05_b012",
  "key": "pressthespacebartocontinue",
  "page": 5,
  "role": "body",
  "translate": true,
  "src": "vector",
  "align": "left",
  "lines": [
    {"bbox": [72.0, 410.2, 301.5, 422.8], "size": 9.5, "font": "serif", "weight": 500, "angle": 0}
  ],
  "text": "Press the SPACE BAR to continue",
  "flow_next": null
}
```

★ 번역은 id가 아니라 원문 키(원문에서 영숫자만 남긴 소문자 문자열)로 붙인다. 재분석하면 블록 번호가 밀려 엉뚱한 상자에 번역이 들어간다.

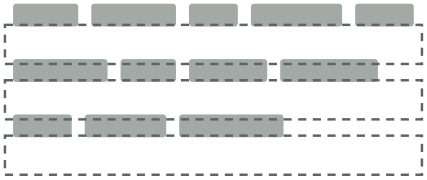
★ 알고리즘을 고친 뒤에는 번역 단위를 처음부터 다시 만들고, 사람이 고친 값만 원문 키로 다시 잇는다. 그대로 두면 옛 계산값이 사람 편집처럼 살아남는다.

– 문서마다 다른 손질은 코드 분기가 아니라 규칙 파일로 둔다. 지우기만 할 영역, 사슬 잇기·끊기, 칸마다 손 배치, 번역 제외.

좌표의 기준은 잉크 행이다

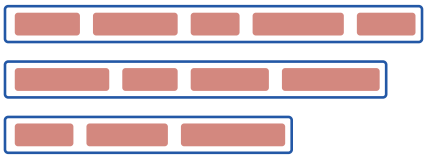
비전 모델이든 OCR API든, 돌려주는 상자는 반 줄에서 한 줄씩 어긋난다. 한 줄을 둘로 쪼개거나 다음 줄에 폭 전체 상자를 붙이기도 한다. 상자로는 글자가 있는 구역만 정하고, 위치·크기·기울기는 이미지에서 직접 잰다.

모델 상자를 그대로 쓰면



상자가 반 줄씩 내려가 있다.
상자로 고르면 윗줄 글자가 아랫줄로 간다.

잉크 행으로 다시 재면



행 상자(파랑)가 글줄에 붙고,
마스크(빨강)는 획만 덮는다.

모델 상자는 구역 힌트로만 쓴다. 구역 안의 잉크 연결 성분을 행으로 묶어 위치·크기·기울기를 다시 재고, 읽은 글은 그 행에 순서대로 붙인다.

잉크 행 만드는 법

- 구역(모델 상자에 세로 2줄·가로 1.5글자 여유) 안을 이진화해 연결 성분을 뽑는다. 연한 색 제목이 빠지지 않도록 문턱은 바탕 밝기 기준으로 넉넉히 잡는다.
- 검은 바탕 흰 글자는 상자 안 중앙값 밝기로 판단해 극성을 뒤집는다.
- 행 묶기는 한 가지 방식만 쓴다. 기울기를 되돌린 좌표에서 1차원으로 묶고, 행 기준 높이를 구한 뒤 성분을 가장 가까운 행에 다시 배정한다. 수평 전용 탐욕 조립은 손글씨 디센더가 아랫줄을 빨아들인다.
- 1차 묶기에는 보통 크기 성분(중앙값의 0.5~1.3배)만 쓴다. 점·따옴표는 줄 사이에 떠서 윗줄과 아랫줄을 사슬처럼 잇는다.
- 기준 높이는 성분 3개 이상인 묶음에서만 구한다. 윗줄 디센더와 아랫줄 어센더가 붙은 성분, 중심이 행에서 글자 높이의 0.6배 넘게 벗어난 성분은 마스크에만 넣고 크기 계산에서 뺀다.
- 마침표 같은 작은 성분은 행의 가로 범위에 성분 중심이 아니라 가장자리 기준으로 붙인다.
- 글자 크기: 잉크 높이는 디센더 유무로 30%씩 흔들린다. 대문자·디센더 유무로 em을 추정하고, 본문은 쪽 중앙값으로 안정화한다.
- 기울기는 구역의 투영 프로파일로 잰다. 여러 줄에 걸친 상자에서 회귀를 쓰면 0°가 나온다. 높이는 상자 모서리가 아니라 픽셀을 되돌려 잰다. 한 행만 각도 추정이 실패하면 구역 각도를 쓴다.
- 삽화 조각 행(높이가 문단 행의 0.4배 미만)과 가는 장식선은 글을 받지 않는다. 글도 없고 높이도 다른 행은 지우지도 않는다. 삽화를 지우는 쪽이 더 나쁘다.

읽은 글을 행에 붙이는 법

- 문단 크롭을 다시 읽혀 줄 목록을 받는다. 14줄 이하로 끊고, 기울어진 문단은 끊지 않고 16줄까지 둔다. 축정렬 크롭을 끊으면 옆줄이 끼어 한 줄로 합쳐 읽는다.
- 줄 수가 행 수와 같고 폭·높이가 그럴듯하면 순서대로 붙인다.
- 다르면 순서를 지키는 DP로 배정한다. 비용은 행 폭 ↔ 글자 수 정합성(글자당 폭은 그 문단에서 실측)에 중심 거리를 약하게 더한 것이고, 한 행이 두 줄을 받을 수 있다. 가장 가까운 상자에 붙이면 한 줄씩 밀린다.
- 글을 못 받은 행은 그 행만 다시 읽는다. 1차 구역 밖의 짧은 한 단어짜리 행(아이콘 무늬를 단어로 읽은 헛글)은 버린다.

글자 읽기 요청 예

```
이 이미지에서 보이는 글자를 줄 단위로 읽는다.
JSON 배열만 출력한다: [{"text": "...", "bbox": [x0, y0, x1, y1]}]
bbox 는 이 이미지 기준 0~1000 정규화 좌표다.
보이는 그대로 옮기고, 고치거나 번역하지 않는다.
그림의 무늬·장식은 글자로 읽지 않는다.
```

번역 규칙

⚠ "간결하게", "짧게"를 지시하지 않는다. 모델이 요약한다. 대신 "요약·생략 금지. 조건·경고·예시·숫자·키·캡션 전부 보존"을 적는다.

– 역할을 함께 준다. 표 칸·키·버튼(label)은 짧은 명사구, 제목(heading)은 명사구, 본문(body)은 문장. 라벨을 본문으로 주면 서술문이 나와 칸에 안 들어간다.

– 길이 목표: 한글은 영문 글자 수의 0.6~0.8배. 벗어나면 플래그만 단다. 조판에서 먼저 줄이고, 그래도 안 되는 블록만 문장을 손본다.

– 단 끝·쪽 끝에서 끊긴 문장은 사슬로 묶어 한 단위로 번역한다. 따로 번역하면 반쪽 문장 두 개가 된다.

– 사슬 판정: 문장부호 없이 끝나는 문단 블록에서, 같은 단 아래 → 다음 단 맨 위 → 다음 쪽 첫 단 맨 위 순으로 소문자로 시작하는 블록을 찾는다. 표 칸도 소문자로 시작하니, 문단처럼 생긴 블록(줄당 평균 4단어 이상, 첫 줄이 최장 줄의 80% 이상)에만 적용한다.

★ 용어집과 문체표를 번역 전에 확정하고, 파일럿 2~3쪽을 사람이 읽은 뒤 전량을 돌린다. 게임 매뉴얼이면 게임 한글패치의 표기가 정본이다.

– 지시문에 예시 용어를 넣었다면, 기존 번역과 다를 때는 기존 번역을 따르게 한다.

– 키보드 키·파일명·제작진 이름은 옮기지 않는다. OCR 오타는 문맥으로 바로잡되, 뜻을 모르면 원문을 둔다.

– 배치를 나누면 표기와 말투가 갈린다. 끝에 표기 목록을 뽑아 용어집 기준으로 통일하는 패스를 둔다. 다수결로 정하지 않는다.

– 검수는 규칙 위반 찾기가 아니라 문장을 다시 쓰는 수준으로 한다.

번역 지시문 (배치 JSON)

```
너는 게임 매뉴얼의 한국어판 번역자다. 입력 JSON 배열의 각 항목을 한국어로 완역한다.

규칙
- 요약·생략하지 않는다. 조건·경고·예시·숫자·키·표 항목·캡션을 모두 옮긴다.
- role=label 은 칸에 들어가는 짧은 명사구, role=heading 은 명사구, role=body 는 문장으로 쓴다.
- 규칙 설명은 "~한다" 체로, 인용된 편지·대사는 화자의 말투로 쓴다.
- 용어는 [용어집]을 따른다. 용어집에 없는 고유명사는 음역하고 notes 에 적는다.
- 키보드 키·파일명·제작진 이름은 영문 그대로 둔다.
- OCR 오타로 보이는 단어는 문맥으로 바로잡고, 뜻을 알 수 없으면 원문 그대로 둔다.
- ctx 는 앞뒤 문맥 참고용이다. 번역하지 않는다.
- 출력은 JSON 배열만: [{"id": "...", "ko": "...", "notes": ""}]. 입력의 모든 id 를 빠짐없이 순서대로.

[용어집]
(원문 = 한국어, 한 줄에 하나)

[입력]
[{"id": "p05_b012", "role": "body", "en": "...", "ctx": "..."}]
```

검수 지시문

```
아래는 매뉴얼의 원문과 번역이다. 원문과 한 항목씩 대조해
오역·누락·뜻이 뒤집힌 곳(can/cannot 등)·어색한 한국어 문장을 찾아 다시 쓴다.
[용어집]과 표기가 다른 곳도 고친다.
고친 항목만 JSON 배열로 출력한다: [{"id": "...", "ko": "고친 번역", "why": "이유"}]

[용어집]
(원문 = 한국어)

[항목]
[{"id": "...", "role": "...", "en": "...", "ko": "..."}]
```

지우기 규칙

- 벡터 글자는 텍스트 객체만 지운다(PyMuPDF `add_redact_annot` 후 `apply_redactions(images=PDF_REDACT_IMAGE_NONE, graphics=PDF_REDACT_LINE_ART_NONE)`). 인페인팅이 필요 없다.
- 래스터 글자의 마스크는 잉크 성분으로 만들고 1~2px만 넓힌다. 상자로 만들면 윗줄 글자와 옆 삽화가 같이 지워진다. 큰 성분은 행의 글자 가로 범위 안으로 자른다.
- 괘선·장식과 붙어 한 덩어리가 된 큰 글자는 통째로 버리지 말고, 구역 안쪽만 잘라 마스크에 넣는다. 절반만 지우면 남은 획을 생성 모델이 글자로 복원한다.

★ 종이·단색 바탕은 고전적인 채움(OpenCV Telea)이 가장 정확하다. 생성형 인페인팅은 망점·질감 바탕에만 쓴다.

- 바탕 판정: 마스크 둘레 띠를 12px 조각으로 나눠 조각별 표준편차를 구하고, 그 중앙값이 18 미만이면 평탄하다. 띠 전체 표준편차로 재면 한 구석의 망점 때문에 오판한다.

⚠ 생성형 모델에 글자가 보이는 이미지를 주면 마스크 안에 글자 비슷한 낙서를 다시 그린다. Telea로 글자를 먼저 없앤 이미지를 주고 질감 복원만 시킨다.

- 모델 결과는 마스크 안만 원본에 합성한다. 마스크 밖이 달라졌거나 채운 자리에서 글자 모양이 다시 읽히면 버리고 Telea 결과를 쓴다.
- 스캔 문서는 줄 상자 단위로 지우면 숨은 층이 빠뜨린 줄이 남는다. 번역 블록 상자(+0.25em) 안에서 바탕보다 충분히 어두운 픽셀(바탕 밝기 80분위 − 45보다 어두운 것)을 지운다.
- 블록 밖에 남은 꼬리 줄은 '지우기만' 규칙으로 사각형을 지정한다.

⚠ 투명 마스크가 있는 이미지를 옮기면 검은 사각형이 된다. 흰 바탕에 알파 합성한다.

⚠ 판독용으로 축소한 이미지를 결과물에 쓰지 않는다. 지우기와 조판은 원본 해상도 좌표에서 한다.

조판 규칙

- 서체 무드는 원본 서체명으로 분류한다(serif·sans·mono·hand·display). 명조는 Noto Serif KR, 고딕·장식체는 Noto Sans KR, 손글씨는 손글씨체. 배포물에는 OFL 서체를 쓴다.
- 굵기는 이름(Bold)이 아니라 획 두께로 고른다. PDF에 내장된 원본 서체의 세로 획 두께/em을 재고, 가변 한글 서체를 300~900으로 인스턴스화해 같은 방법으로 잰 뒤 가장 가까운 값을 쓴다.
- 크기: 한글은 같은 pt에서 커 보인다. 0.94배로 시작해 줄 수를 지키며 0.88까지 줄이고, 그래도 넘치면 줄 간격을 줄여 줄을 더 만든다. 하한은 0.72이고, 거기서도 넘치면 플래그를 단다.

⚠ 블록마다 자기 최대 크기에서 자동 축소하지 않는다. 같은 쪽에서 크기가 들쭉날쭉해진다. 쪽 묶음별 기준 크기를 정하고 모자랄 때만 줄인다.

⚠ 스캔의 숨은 층이 알려 주는 글자 크기는 실제보다 작다. 베이스라인 간격과 비교해 보정한다(대개 1.2배 안팎).

- 줄 폭은 줄마다 원본 폭을 쓴다. 그래야 삽화를 돌아 흐르는 문단도 원래 실루엣대로 흐른다.
- 사슬 조판: 구성원 블록의 줄 상자를 순서대로 이어 붙인 프레임에 흘린다. 앞 구간은 원본 줄 상자만 쓰고 넘침은 마지막 구간에서만 처리한다. 단어가 이번 줄 상자에 안 들어가면 줄을 비우고 다음 상자로 넘긴다. 구성원 블록마다 자기 정렬을 쓴다.
- 굵기는 스팬 단위다. 굵은 제목이 섞였다고 블록 전체를 굵게 하지 않는다. 연도·주문명 같은 머리 라벨은 번역문의 접두부·첫 어절만 굵게 한다.
- 정렬: 한 줄짜리 블록은 혼자 판정할 수 없으니 같은 단 이웃을 따른다. 가운데 정렬은 양쪽 가장자리가 다 들쭉날쭉할 때만 준다. 곧은 가장자리는 블록 중앙값에 맞춰 붙인다. 원본 단어 상자의 x가 줄마다 흔들린다.
- 기울어진 글은 수평 프레임에서 조판한 뒤 원래 각도로 되돌린다(PyMuPDF `TextWriter.write_text(morph=…)`).

표와 지도

- 지도는 글자가 아니라 기하로 판정한다. 제목 아래 행 대부분이 단 폭의 절반 미만이고 본문 폭 행이 하나 이하면 지도다.
- 평문 추출에서 지도 라벨 뒤에 붙은 문장은 대개 직전 본문의 이어짐이다.

⚠ 숫자 표를 추출 행으로 재구성하면 숫자가 한 줄씩 밀린 틀린 표가 된다. 괘선 기하로 셀을 잡는다. 칸이 적으면 칸마다 손으로 배치하고, 어느 쪽도 안 되면 원문을 유지한다.

- 표의 키 글자 한 자는 반 칸만 떨어져도 다른 칸이다. 글자 간격을 벌린 단어(`r i g h t`)는 한 단어로 합친다.
- 삽화 속 아주 작은 글자(본문 크기의 45% 미만)는 번역하지 않는다. 지우면 삽화가 상한다.

검수와 납품

자동 검사

- [ ] 번역 대상 블록 전부에 번역이 있고 빈 값이 0이다
- [ ] 혼입 문자(한자·중국어·깨진 글자·반쪽 영단어)가 0이다
- [ ] 숫자·키 글자·주사위 표기(1d6)가 원문과 같다
- [ ] 조판 넘침이 0이고, 축소율 0.85 미만 블록 목록이 나왔다
- [ ] 번역 블록끼리 상자 겹침 쌍 수가 이전 판보다 줄었다

눈으로 — 원본|결과 나란히, 전 쪽

영문 잔존 · 글 겹침 · 잘린 꼬리 · 빠진 문단 · 검은 사각형 · 삽화 훼손 · 읽기 순서 · 크기 들쭉날쭉 · 제목 위치

- 비전 모델에 쪽 구간별로 원본·결과 이미지를 주고 위 범주로 문제 목록(JSON: 쪽·위치·범주·설명)을 받는다. 목록은 한 건씩 고치지 말고 원인별로 묶어 코드나 규칙으로 고친다.
- 작은 글자·기울어진 글·검은 바탕 흰 글자·삽화 인접 영역은 확대로 다시 본다.
- 인쇄 쪽 번호와 PDF 쪽 번호가 다르면(펼침면 스캔) 환산식을 적어 둔다.
- 판마다 파일명에 버전을 넣고 해시·옵션·남은 결함을 기록한다. 이전 판은 지우지 않는다.

⚠ 매뉴얼 문장과 삽화의 저작권은 원저작권자에게 있다. 번역본은 개인 참고용으로 두고, 공개 배포는 허락을 받는다.

방식 1

로컬 AI 방식 — GPU 한 대로 끝내기

외부 API 없이 한 PC에서 끝낸다. 비용은 전기뿐이고 문서가 밖으로 나가지 않는다. 대신 모델 품질의 한계와 VRAM·ollama 관리가 노하우의 대부분이다.

작업 지시서 — 로컬 AI

역할	영문 PDF(게임 매뉴얼·저널·레퍼런스 카드)를 한국어판으로 재조판하는 작업자
목표	원본 쪽의 위치·크기·굵기·정렬·삽화를 유지하고, 영문만 한국어 실제 텍스트로 바꾼 PDF
도구	Python(PyMuPDF·OpenCV·numpy·fontTools), ollama(비전 모델·번역 모델), 선택으로 ComfyUI(이미지 편집 모델). 외부 API는 쓰지 않는다
입력	원본 PDF, 용어집·문체표(없으면 초안을 만들어 확인받는다), 결과물 형태(원본 위 재조판 / 새로 조판)
진행	공통 원리의 7단계를 순서대로 진행한다. 단계마다 JSON과 오버레이 이미지를 남기고 게이트를 확인한 뒤 넘어간다. 모든 모델 호출은 캐시한다
멈추고 확인	단 경계 확정 · 결과물 형태 · 용어집과 파일럿 번역 · 표 쪽 처리 방침 · 최종 전 쪽 검수
금지	원본 덮어쓰기 · 한글을 이미지로 넣기 · 글자가 보이는 이미지를 생성 모델에 넣기 · ollama 작업 동시 실행 · 전 쪽 비교 없이 완료 선언

구성

역할	로컬 구성	핵심 설정
글자 읽기	ollama 비전 모델 예: qwen3-vl 4B instruct	temperature 0, JSON 출력, 상자 좌표 0~1000 정규화, 타일 한 변 1400px 이하
번역 초벌	ollama 27B급 번역 모델 예: qwen 27B Q4	temperature 0, JSON 출력, 생각 모드 끔, 단위별 캐시
바탕 채움	OpenCV Telea(기본) + ComfyUI 이미지 편집 모델 예: Qwen-Image-Edit + Lightning 4-step	편집 모델은 질감 바탕에만
지우기·조판	PyMuPDF + fontTools	redaction, TextWriter, 가변 서체 굵기 인스턴스
검수	PyMuPDF 렌더 비교 시트 선택: 비전 모델	원본|결과 나란히, 쪽별

VRAM 기준: 비전 4B(약 5.5GB)와 번역 27B(약 19GB)는 함께 올라간다. 편집 모델(fp8 약 20GB + 텍스트 인코더 약 8GB)은 둘과 번갈아 쓴다.

비전 모델로 글자 읽기

★ 쪽을 통째로 주지 않는다. 한 변 1400px 이하 타일에 12% 겹침. 큰 펼침면을 통째로 주면 작은 라벨을 빠뜨리고 긴 쪽의 뒷부분을 잘라 먹어, 줄의 절반 이상이 사라진다.

– 1차 패스는 구역 찾기다. 타일마다 `{text, bbox}` 목록을 받아 잉크 행 구역만 정한다.

- 2차 패스는 문단 크롭 다시 읽기다(14줄 이하, 공통 원리의 배정 규칙). 한 단을 통째로 주면 줄 수를 모자라게 돌려준다.
- 좁은 단이 붙어 있으면 타일 OCR이 옆 단 글을 섞는다. 행 폭 대비 글자 수가 중앙값의 1.5배를 넘는 행이 보이면 그 쪽은 단 크롭으로 다시 읽는다.
- 시간 초과로 비는 타일이 생긴다. 빈 행·빈 단만 골라 다시 읽는다.
- 모델 응답은 캐시한다. 후처리만 바꿨을 때는 모델을 다시 부르지 않고 캐시로 재처리한다.

ollama 운용

⚠ 컨텍스트 길이(`num_ctx`)는 병렬 슬롯(`OLLAMA_NUM_PARALLEL`)이 나눠 갖는다. 16k에 슬롯 4개면 슬롯당 4k라 이미지 요청이 넘쳐 연결이 끊긴다. 이미지 요청은 65k로 준다.

⚠ `num_ctx`는 요청마다 같은 값으로 보낸다. 값이 바뀌면 모델을 다시 올리고, 그동안 들어온 요청은 끊긴다.

⚠ ollama 작업은 한 번에 하나만 돌린다. 두 작업이 겹치면 500 오류와 시간 초과가 난다. GPU를 쓰는 다른 프로그램(게임 등)을 띄우면 런너가 내려가 요청이 매달린다.

- 요청 시간 제한은 600초로 짧게 둔다. 기본값이면 매달린 요청 하나에 30분이 멈춘다.
- 실패한 요청은 그 자리에서 붙잡지 말고 건너뛴다. 단위별 캐시가 있으니 전체가 끝난 뒤 빠진 단위만 다시 돌린다. 런너가 간헐적으로 끊겨도 이 방식이 가장 빠르다.
- 시스템 프롬프트는 5KB 이하로 압축한다. 긴 스타일 가이드를 통째로 넣으면 컨텍스트를 넘기거나 규칙을 놓친다. 용어집은 그 배치에 나오는 항목만 넣는다.

★ VRAM 교대: 번역·OCR을 시작하기 전 ComfyUI 모델을 내리고(`POST /free`), 인페인팅 전에는 ollama 모델을 내린다(`keep_alive: 0`). 안 내리면 27B가 CPU로 밀려 20배 느려진다.

로컬 모델로 번역하기

★ 로컬 27B급은 설명서체(규칙·기능 설명·표·라벨)에는 충분하다. 이야기·편지·대사·말장난은 문장과 말투의 한계가 낮다. 그 부분은 상위 모델이나 사람에게 넘긴다.

⚠ 로컬 모델에게 번역문 다듬기·재작성을 시키지 않는다. 절반 이상을 그대로 돌려준다. 고칠 줄은 사람이나 상위 모델이 고친다.

- temperature는 0. 올리면 품질이 떨어진다. JSON 출력 모드를 켜고 생각 모드는 끈다.
- 한 요청에 10~25단위를 넣는다. 입력은 `{id, role, en, ctx}`이고, ctx에는 앞뒤 블록 한 줄씩과 쪽 제목을 넣는다. 지시문은 공통 원리의 번역 지시문을 쓴다.
- 출력을 검증하고 실패한 단위만 재요청한다. id 누락, 빈 값, 혼입 문자(한자·중국어·반쪽 영단어), 숫자·키 불일치, 길이 비율 이탈.
- 속도 기준: 27B 번역 약 190단위에 2분, 비전 모델 OCR 18쪽에 약 4분(캐시 재처리는 쪽당 수 초).

ComfyUI로 바탕 채우기

1 잉크 마스크를 1~2px 넓힌다.

2 OpenCV Telea로 먼저 채운다. 글자는 여기서 사라진다.

3 바탕 판정(조각별 표준편차 중앙값 < 18). 평탄하면 Telea 결과로 끝낸다.
종이 바탕 매뉴얼은 대부분 여기서 끝난다.

4 질감 바탕만, 마스크 둘레에 여유를 둔 크롭을 Telea 결과로 만들어 편집 모델에 준다. 지시는 "주변과 같은 질감으로 채운다. 글자·기호를 그리지 않는다".

5 결과를 원래 크기로 되돌려 마스크 안만 합성한다.
마스크 밖이 바뀌었거나 글자 모양이 보이면 버리고 Telea 결과를 쓴다.

– Lightning 4-step이면 호출당 4초 안팎이다. 평탄 바탕을 먼저 거르면 호출 수가 크게 준다.

⚠ 편집 모델을 올린 채로 번역을 돌리지 않는다(ollama 운용의 VRAM 교대 참고).

방식 2

API 방식 — 로컬 AI 없이 클라우드로

로컬 GPU가 없으면 글자 읽기·번역·인페인팅을 클라우드 API나 서비스로 바꾼다. 좌표 계산·지우기·조판은 AI가 아니라 일반 스크립트(PyMuPDF·OpenCV)라 어디서나 돈다. 공통 원리는 그대로이고, 달라지는 것은 비용·해상도 제한·문서 업로드다.

작업 지시서 — API

역할	영문 PDF(게임 매뉴얼·저널·레퍼런스 카드)를 한국어판으로 재조판하는 작업자
목표	원본 쪽의 위치·크기·굵기·정렬·삽화를 유지하고, 영문만 한국어 실제 텍스트로 바꾼 PDF
도구	Python(PyMuPDF·OpenCV·numpy·fontTools), 문서 OCR API 또는 비전 LLM API, 텍스트 LLM API, 마스크 인페인팅 API(선택). 로컬 AI 모델은 쓰지 않는다
입력	원본 PDF, 용어집·문체표(없으면 초안을 만들어 확인받는다), 결과물 형태, 사용할 API와 예산
진행	공통 원리의 7단계를 순서대로 진행한다. 벡터 글자 쪽은 OCR API를 부르지 않는다. 모든 API 호출은 원문 키·이미지 해시로 캐시한다
멈추고 확인	단 경계 확정 · 결과물 형태 · 용어집과 파일럿 번역(모델 선택 포함) · 인페인팅 대상 영역 수와 예상 호출 수 · 최종 전 쪽 검수
금지	원본 덮어쓰기 · 한글을 이미지로 넣기 · 글자가 보이는 이미지를 생성 모델에 넣기 · API가 돌려준 축소 이미지를 원본 대신 쓰기 · 권리 확인 없는 문서 업로드 · 전 쪽 비교 없이 완료 선언

로컬 구성을 무엇으로 바꾸나

역할	로컬 AI 방식	API 방식	주의
글자 읽기	ollama 비전 모델	문서 OCR API(단어 좌표 제공) 또는 비전 LLM API	좌표는 OCR API가 더 정확하고, 글 교정은 LLM이 낫다
번역	로컬 27B	텍스트 LLM API	배치·캐시로 비용을 관리한다
바탕 채움	Telea + ComfyUI 편집 모델	Telea + 마스크 인페인팅 API	평탄 바탕은 호출하지 않는다
지우기·조판	PyMuPDF	PyMuPDF (같다)	AI가 아니라 스크립트
검수	렌더 시트 + 비전 모델	렌더 시트 + 비전 LLM API	쪽 구간별로 나눠 보낸다

글자 읽기

★ 벡터 글자 쪽(A)은 OCR을 부르지 않는다. PDF가 가진 글자와 좌표를 그대로 쓴다. 비용이 가장 크게 줄어드는 곳이다.

– 좌표가 필요하면 단어 상자를 돌려주는 문서 OCR API(예: Google Cloud Vision, Azure AI Document Intelligence, Amazon Textract)를 먼저 쓴다. 비전 LLM의 상자보다 위치가 정확하다. 그래도 최종 위치는 잉크 행으로 보정한다.

– 비전 LLM으로 읽을 때는 쪽 이미지를 직접 잘라 보낸다(한 변 1400~1600px, 12% 겹침). API가 큰 이미지를 내부에서 줄이면 작은 글자가 사라진다.

- 좌표계를 지시문에 못박고(크롭 기준 0~1000 정규화 등), 출력은 JSON 스키마로 강제한다. 구조화 출력 기능이 있으면 쓴다.
- 출처를 섞지 않는다. OCR API 결과, LLM 결과, 숨은 층 중 단마다 하나를 정본으로 삼고, 다른 출처는 글 내용 교정에만 참고한다.
- PDF 파일을 통째로 받는 API도 있지만, 재조판에는 좌표가 필요하므로 쪽 이미지와 크롭을 쓴다.

LLM API로 번역하기

★ 모델은 파일럿으로 고른다. 2~3쪽을 후보 모델로 번역해 사람이 읽는다. 이야기·대사·말투는 상위 모델, 표·라벨·단순 규칙 문장은 저가 모델로도 된다.

- 시스템 프롬프트(공통 원리의 번역 지시문 + 용어집)는 고정하고, 요청마다 배치만 바꿔 넣는다. 프롬프트 캐싱을 지원하면 고정부가 캐시돼 비용이 준다.
- 배치는 영어 2,000단어 안팎. 크게 넣으면 뒤쪽을 줄이거나 빠뜨린다.
- 출력은 `{id, ko}` JSON 배열로 강제하고 검증한다. 모든 id가 있는지, 빈 값·혼입 문자가 없는지, 숫자·키가 일치하는지, 길이 비율이 0.6~0.8인지 본다. 실패한 id만 다시 보낸다.
- 급하지 않은 대량 번역은 비동기 배치 API를 쓴다. 대개 더 싸다. temperature는 낮게 둔다.
- 원문 키로 캐시한다. 재분석·재조판을 여러 번 해도 이미 번역한 단위는 다시 보내지 않는다.
- 병렬 요청을 쓰면 표기가 갈린다. 끝에 표기 목록을 뽑아 용어집 기준으로 통일한다.
- 검수 패스는 별도 요청으로 돌린다(공통 원리의 검수 지시문). 가능하면 번역보다 상위 모델을 쓴다.

인페인팅 API·서비스로 바탕 채우기

종류	입력과 동작	쓰는 법
마스크 인페인팅 API 예: Stability AI Inpaint, FLUX Fill 계열	이미지 + 마스크 → 마스크 안만 다시 그린다	권장. 크롭 단위로 보낸다
대화형 이미지 편집 모델 예: ChatGPT·Gemini 이미지 편집	이미지 + 지시문 → 이미지 전체를 다시 생성한다	선·색·해상도가 바뀔 수 있다. 결과는 마스크로 합성해서만 채택한다
편집기 생성형 채우기 예: 포토샵 생성형 채우기	선택 영역 → 채움	사람이 손으로. 영역이 적을 때

1 잉크 마스크 → 1~2px 넓히기 → Telea 채움. 글자는 여기서 없앤다.

2 바탕 판정. 평탄하면 끝낸다.
API를 부르지 않는다.

3 질감 바탕만 크롭(마스크 둘레 여유 32~64px)으로 잘라, Telea 결과 크롭과 마스크를 보낸다.

4 지시문: "주변과 같은 종이·망점 질감으로 채운다. 글자·기호·무늬를 새로 그리지 않는다."

5 결과가 다른 크기로 오면 원래 크기로 되돌리고, 마스크 안만 원본에 합성한다.

6 검사: 마스크 밖 차이 0, 채운 자리 재OCR에서 글자 검출 0, 색·알파 유지.
실패하면 Telea 결과를 쓴다.

⚠ 글자가 남은 이미지를 보내지 않는다. "글자를 지워라"라고 해도 생성 모델은 글자 비슷한 획을 다시 그린다.

⚠ 쪽 전체를 보내고 돌려받은 쪽을 통째로 쓰지 않는다. 해상도 제한 때문에 줄어들거나 삽화 선이 바뀐다.

- 비용 관리: 대상 영역 수와 예상 호출 수를 먼저 뽑아 확인받는다. 종이 바탕 문서는 대부분 호출이 필요 없다.
- 업로드 전에 문서의 권리와 기밀 여부, 서비스 약관의 데이터 사용 조건을 확인한다.

코딩 없이 할 때

스크립트를 못 돌리면 같은 규칙을 손으로 적용한다. 원본 위 재조판은 쪽 수가 적을 때만 현실적이고, 대개는 새로 조판이 낫다.

- 번역: 채팅 AI에 쪽 단위(영어 2,000단어 안팎)로 공통 원리의 번역 지시문과 용어집을 붙여 넣는다. JSON 대신 원문을 붙이고 출력 형식을 "번역문만"으로 바꿔 쓴다. 단 끝·쪽 끝에서 끊긴 문장은 다음 문단과 붙여서 준다.
- 스캔 쪽: 쪽 이미지를 한 쪽·반쪽씩 올려 "보이는 그대로 옮겨 적기"를 먼저 시키고, 번역은 따로 시킨다.
- 텍스트 PDF 교체: Acrobat Pro의 PDF 편집으로 글상자 안 문장을 바꾼다.
- 그림 속 글자: 이미지 편집기에서 복구 브러시·도장으로 먼저 지우고, 질감 바탕만 생성형 채우기를 쓴다. 선택 영역은 글자 획만 잡는다.
- 새로 조판: 워드·한글·Google 문서에서 원본 판형을 맞추고 삽화를 넣는다. 본문 크기와 줄 간격은 하나로 고정한다.
- 검수: 원본·결과 쪽 이미지를 비전 AI에 올려 검수 범주별 문제 목록을 받는다.

공통 원리의 수치(0.94배, 14줄, 1400px, 표준편차 18 등)는 실제 매뉴얼·저널 재조판 작업에서 잰 값이다. 문서에 따라 파일럿으로 조정한다.